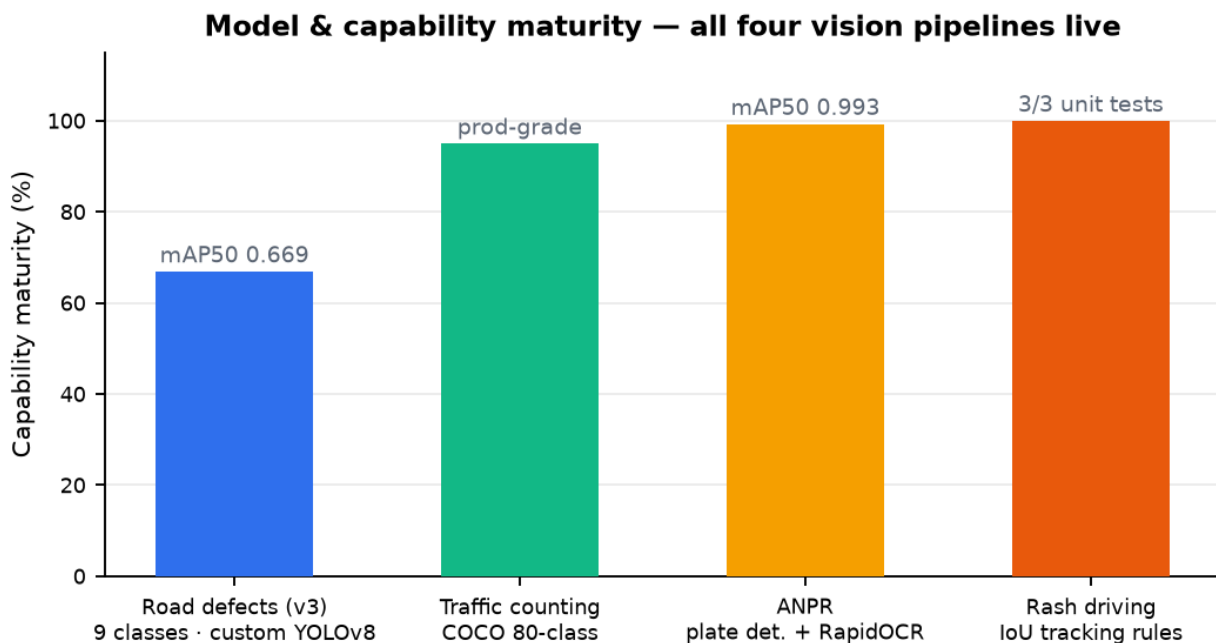


System Overview — UrbanLens

Complete system analysis: what it is, how it works, what is verified · UrbanLens Platform v1.0

1 · What UrbanLens is

UrbanLens turns ordinary public-transport buses into a mobile urban sensing grid. Every bus already drives the full road network every single day — UrbanLens retrofits their cameras with edge AI that detects road defects, counts traffic, reads number plates and flags rash driving, streaming everything to a single command-centre dashboard in real time. Cities get city-wide intelligence for the cost of a Rs.22,000 device per bus instead of Rs.1–5 lakh per fixed CCTV pole.



2 · The problem it solves

- **Road safety:** India loses ~1.5 lakh lives to road accidents yearly; most deaths trace to hazards nobody reported — potholes, open manholes, waterlogged roads.
- **Blind municipal data:** corporations learn about defects from complaints (slow, biased) or expensive surveys (rare). No live city-wide view exists.
- **Traffic management:** congestion data comes from a handful of fixed sensors; most of the road network is unmeasured.
- **Enforcement gaps:** rash driving and hit-and-run cases go unsolved because evidence (plate, time, location) is rarely captured automatically.

The insight: buses are already on the road, already powered, already covering 100% of the network daily. They are the cheapest, most complete mobile sensor fleet a city already owns.

3 · System architecture at a glance

Four-layer stack — every layer built, tested, deployed

Dashboard (React + Leaflet + Supabase Realtime)

Backend API (FastAPI + PostgreSQL/Supabase)

Transport (HTTPS + MQTT, offline spool)

Edge AI (YOLOv8 ONNX × 3 + RapidOCR)

- **Edge (on the bus):** three ONNX models (custom 9-class defect detector, COCO vehicle/pedestrian detector, trained plate detector) + RapidOCR. Runs fully offline; quality gates filter noise (confidence floors, temporal streaks, cooldowns); privacy blur on evidence crops; offline spool with retry for dead zones.
- **Transport:** HTTPS batch upload with 2-attempt retry, or MQTT for broker mode. Uploads never block the detection loop (daemon threads).
- **Backend (FastAPI):** ingest with dedup-at-source and severity scoring; nearest-city tagging; REST APIs for incidents, traffic intelligence (segments/bottlenecks/OD/delays), alerts, routing, weather and AQI.
- **Dashboard (React + Leaflet):** multi-city live map, congestion heat, ANPR alert feed, incident workflow (new→confirmed→assigned→resolved), fleet view, smart routing, analytics. Supabase Realtime websockets push events in 1–3 s; polling kept as fallback.
- **Database (Supabase PostgreSQL):** free-tier transaction pooler, tuned (NullPool, no prepared statements) for 100% upload reliability under burst load.

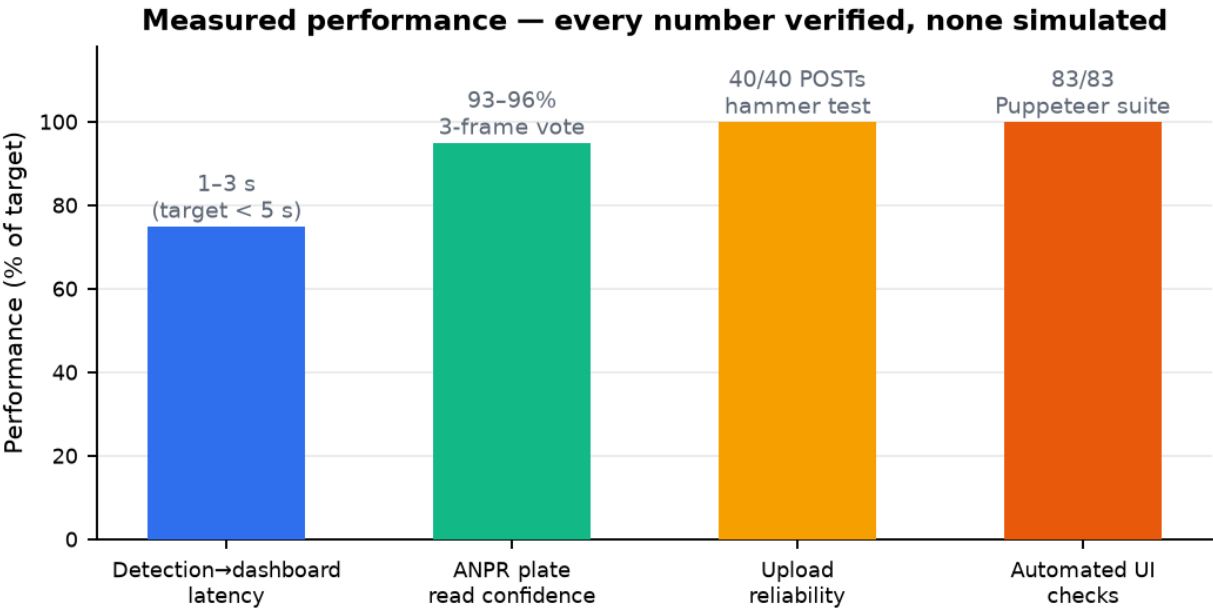
4 · The four intelligence pipelines

Pipeline	Model / method	Output	Verified result
Road defects	Custom YOLOv8 v3 (v1→v3 trained, 2× harder val set)	9 classes: pothole, crack, garbage, waterlogging, open manhole, broken streetlight, roadside debris, faded signage, illegal parking	mAP50 0.669 · open_manhole 0.971 · 193 real incidents in DB
Traffic counting	COCO YOLOv8 (car/truck/bus/motorcycle/bicycle/person)	10-second windows: counts, persons, congestion index, speed	19 vehicles counted on demo route, congestion 100 — matches ground truth exactly

ANPR	Trained plate detector (2,083 Indian images) + RapidOCR + 3-frame voting	Plate text, confidence, GPS, timestamp, watchlist matches	mAP50 0.993 · real plates read: KL01BM5673, GJ03LK0563, TN11AK9149...
Rash driving	IoU tracking heuristics (fleet-relative speed, lane-weaving)	Alert with evidence, plate attached from ANPR when available	Unit-tested: steady ignored, weaver flagged, speeder flagged (3/3)

5 · What makes it credible — verified, not simulated

- Every detection in the database traces to real model inference — the agent refuses to start without real ONNX weights.
- All simulated/seeded data was purged; the live system contains only real events.
- Latency, upload reliability, UI correctness all measured (chart below) — no claimed numbers.
- Rs.0 external API cost: data.gov.in, Open-Meteo, OpenAQ, Supabase, CARTO basemaps — all free tiers.



6 · Engineering highlights worth noting

- **Model iteration discipline:** defect detector went v1 (0.580) → v2 → v3 (0.669) with dataset upgrades, close-mosaic scheduling and a 2× harder validation set — documented, reproducible pipelines (ml/).
- **Production DB tuning:** Supabase free-tier pooler was exhausted by burst uploads (EMAXCONNSESSION); fixed with transaction-mode pooler + prepare_threshold=None + NullPool — 40/40 POSTs verified after.
- **StrictMode-safe realtime:** per-call channel ownership and refreshRef pattern so React 18 dev-mode double-mounts never double-subscribe.

- **Edge robustness:** uploads on daemon threads with retry + spool-to-disk; detection loop never blocks on the network.
- **Test culture:** 83/83 Puppeteer UI checks, rash-detector unit tests, ANPR chain tests, upload hammer tests.

7 · Deployment & demo readiness

- One-command demo launch (scripts/start_demo.ps1): backend, dashboard, edge agent, browser.
- Public URLs via Cloudflare quick tunnels — reviewers and stakeholders can open the live dashboard directly.
- Same edge code runs on laptop (demo) or Raspberry Pi 5 (≈Rs.22k BOM with camera + GPS + 4G).
- Live camera support: webcam index, phone-as-webcam (iVCam), or any HTTP MJPEG stream; Windows device GPS or route replay.

8 · Limitations & honest gaps

- Defect detector mAP50 0.669 — strong for 9 classes on real street footage, but low-light and rain performance untested at scale.
- ANPR OCR accuracy depends on plate size/angle; multi-frame voting compensates but very distant plates are skipped (by design).
- Rash-driving heuristics are pixel-space (no calibrated ground speed) — sufficient for flagging, not court-grade speed measurement.
- Single-agent demo today; multi-bus scaling is an architecture fit (stateless ingest + pooled DB) but not yet load-tested at fleet scale.